INSTITUTO FEDERAL
Rio Grande do Sul

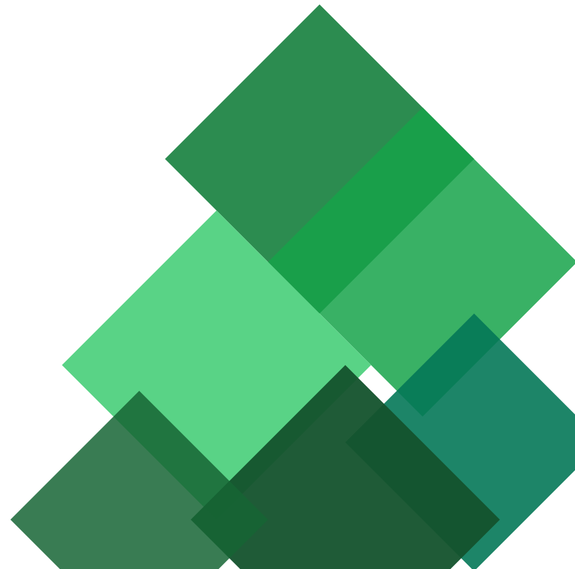
ARQUITETURA DE BACK-END & ENGENHARIA DE DADOS

Comunica Sertão

Desenvolvimento de Serviços Web, Persistência Relacional e Integração Municipal

Autor: Fabricio Cardoso de Abreu

Orientadora: Caroline Guterres Silva



Contexto e Justificativa da Solução

Dos Gargalos Operacionais à Arquitetura Proposta

Cenário e Problema

- **Dispersão Geográfica:** Sertão/RS possui extensa área rural e baixa densidade, dificultando o deslocamento presencial à prefeitura;
- **Descentralização Transacional:** Ausência de uma API centralizada para recepcionar e registrar solicitações de forma unificada;
- **Falta de Rastreabilidade:** Atendimentos informais sem emissão de protocolos oficiais e sem controle de prazos e histórico.

Justificativa Técnica

- **Processamento Assíncrono:** API RESTful com Python e FastAPI, garantindo tempos de resposta instantâneos via webhooks;
- **Persistência Relacional ACID:** Banco SQLite gerenciado com SQLAlchemy ORM, assegurando integridade e confiabilidade de dados;
- **Soberania e Custo Zero:** Código aberto sem cobrança de mensalidades por habitante e custódia segura dos dados dos munícipes.

Objetivos de Desenvolvimento da API

Metas Estruturantes da Camada de Back-End

1. Barramento RESTful (FastAPI)

Prover endpoints padronizados e documentados para integração entre o cidadão e o painel administrativo da prefeitura.

2. Motor de Webhooks e Bot

Receber mensagens em tempo real da API do WhatsApp, controlando a máquina de estados conversacional e a triagem de categorias.

3. Persistência Relacional (SQLite)

Garantir armazenamento atômico (ACID) das ocorrências, munícipes e evidências, facilitando rotinas de backup institucional.

4. Governança e Protocolos Oficiais

Emitir protocolos anuais sequenciais (CMS-2026-XXXX) e gerenciar fluxo de aprovação de acessos restrito a Administradores.

Análise Comparativa de Arquiteturas

Sistemas Existentes vs. Solução Proposta

Plataforma / Solução	Protocolo de Integração	Modelo de Armazenamento	Vantagens Técnicas	Desvantagens / Limitações
Fala.BR (CGU)	Formulários Web / SOAP	Banco relacional centralizado (DF)	Padrão federal e auditoria rígida	Complexo para pequenos municípios; sem mensageria ágil
Colab.re	App Móvel / REST fechada	Nuvem multi-tenant comercial	Zeladoria urbana consolidada	Cobrança contínua por habitante; banco proprietário
Octadesk API	Webhooks / REST comercial	Nuvem sob assinatura SaaS	Roteamento multiatendente	Custo mensal por operador; regras comerciais e não públicas
Comunica Sertão (Back-End)	Webhooks Meta + FastAPI REST	SQLite Local (comunicasertao.db)	Código aberto, baixa latência, soberania e custo zero	Depende da disponibilidade da infraestrutura do host

Requisitos do Sistema

Funcionalidades Essenciais (RFs) e Critérios de Qualidade (RNFs)

Requisitos Funcionais (RFs)

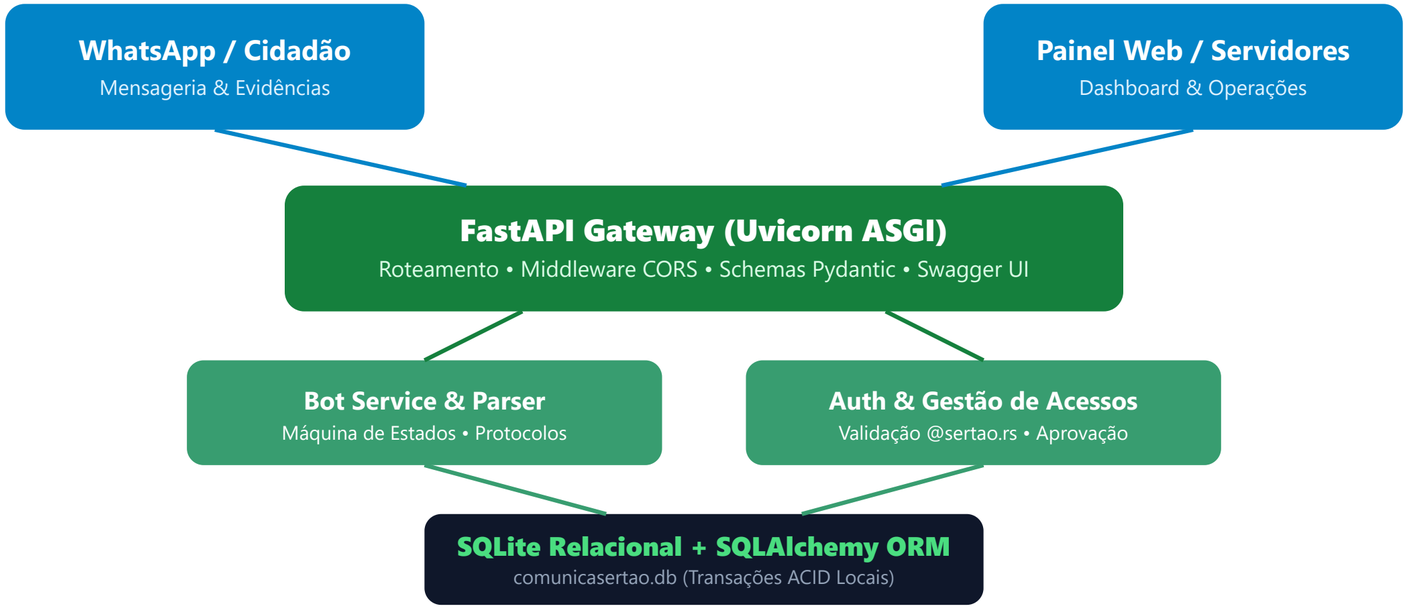
- **RF01 - Recepção Webhook:** Processar mensagens do WhatsApp com parser automatizado;
- **RF02 - Geração de Protocolos:** Criar identificador anual sequencial único por chamado;
- **RF03 - Consulta em Tempo Real:** Retornar andamento da ocorrência por código;
- **RF04 - CRUD de Ocorrências:** Listar, filtrar e atualizar status operacionais;
- **RF05 - Gestão de Acessos:** Autenticação e aprovação de novos servidores.

Requisitos Não Funcionais (RNFs)

- **RNF01 - Baixa Latência:** Resposta às rotas em menos de 200ms com FastAPI e ASGI assíncrono;
- **RNF02 - Integridade ACID:** Banco relacional SQLite local gerenciado com SQLAlchemy ORM;
- **RNF03 - Validação Forte:** Sanitização e validação de contratos de dados com Pydantic;
- **RNF04 - Documentação Aberta:** Especificação OpenAPI e interface Swagger UI nativas;
- **RNF05 - Segurança:** Restrição a e-mails institucionais oficiais (@sertao.rs.gov.br).

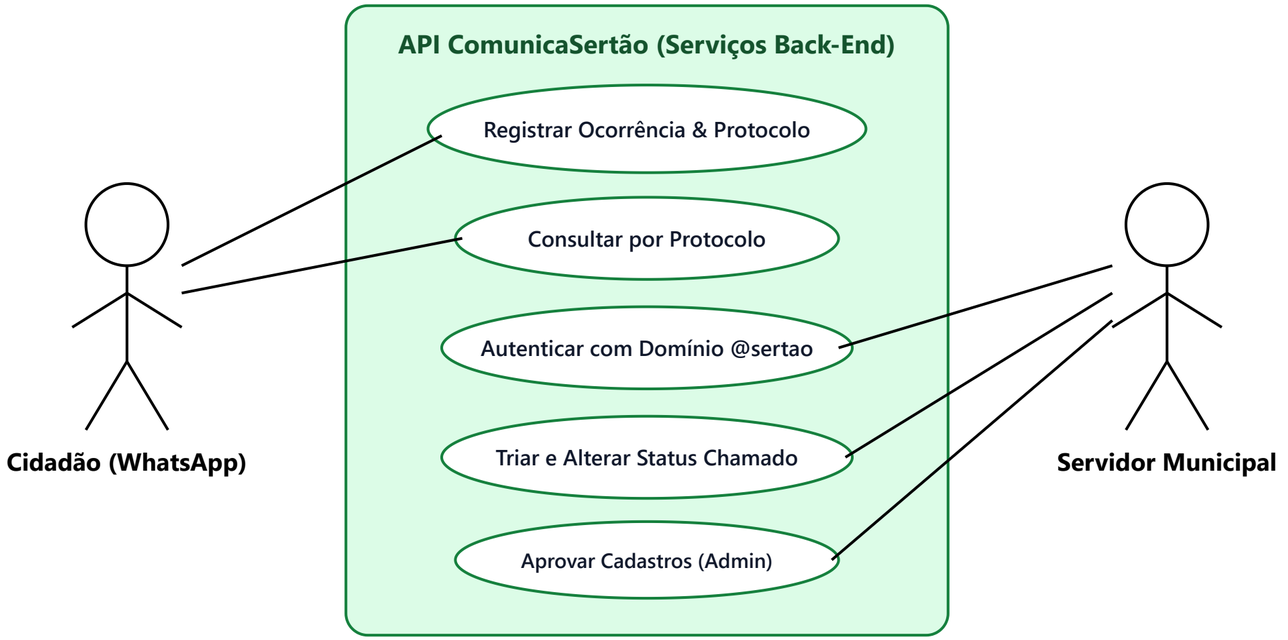
Visão Geral da Arquitetura

Arquitetura em Camadas Orientada a Serviços RESTful



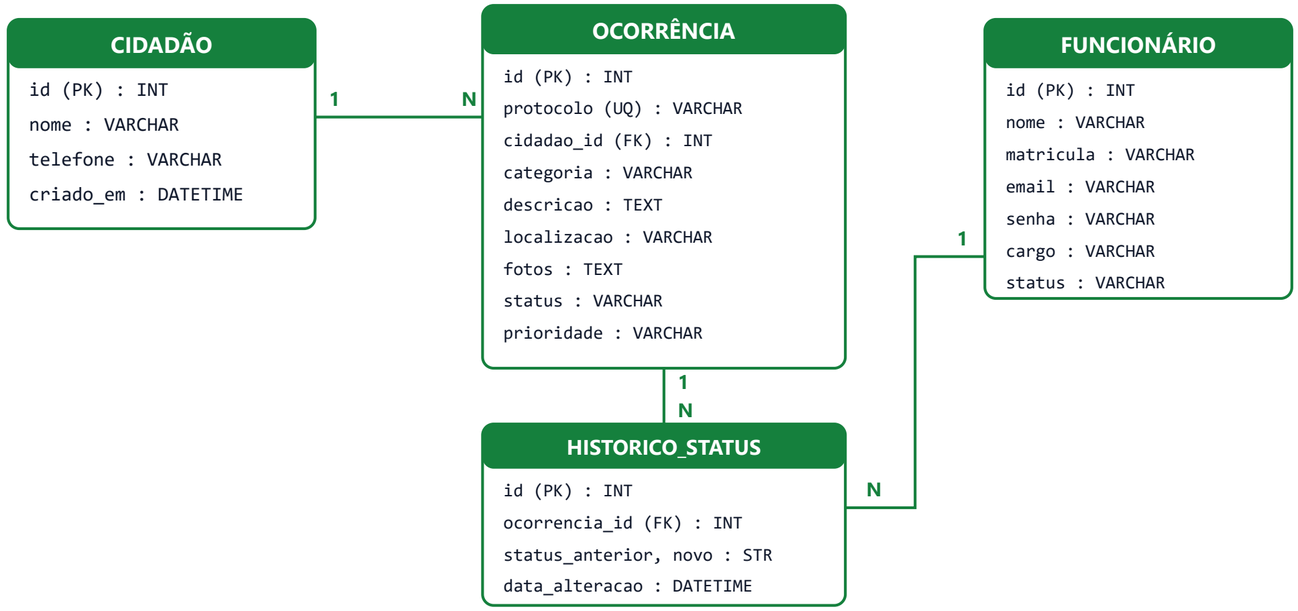
Mapeamento de Casos de Uso (UML)

Interações entre Cidadão, Servidor e API



Modelo de Dados Relacional (DER)

Estrutura do Banco SQLite Gerenciado por SQLAlchemy ORM



Pilha Tecnológica: Execução e Concorrência

FastAPI, Python 3 e Servidor ASGI Uvicorn

FastAPI

- Framework assíncrono moderno desenhado sob o padrão ASGI, permitindo o tratamento de requisições web com concorrência real (async/await);
- Geração automática das especificações OpenAPI 3.0 e UI interativa Swagger;
- Tempos médios de resposta abaixo de 50ms, atendendo com folga ao RNF01.

Python 3 + Uvicorn

- Linguagem de alto nível com suporte a tipagem estática e manutenibilidade limpa;
- Servidor ASGI Uvicorn otimizado para laços de eventos de alto rendimento;
- Baixo consumo de memória RAM, ideal para hospedar em infraestruturas modestas.

Pilha Tecnológica: Dados e Validação

SQLAlchemy ORM, Banco SQLite e Schemas Pydantic

SQLAlchemy + SQLite

- Mapeamento Objeto-Relacional que abstrai comandos SQL e padroniza as entidades;
- Banco SQLite embutido (comunicasertao.db), dispensando processos de banco externos;
- Garantia de propriedades ACID e backups instantâneos por cópia de arquivo.

Pydantic Schemas

- Validação e higienização estrita de todos os payloads de entrada e saída;
- Prevenção contra injeções e bloqueio de requisições com tipos corrompidos;
- Conversão automática de objetos relacionais em respostas JSON tipadas.

Implementação: Documentação Interativa

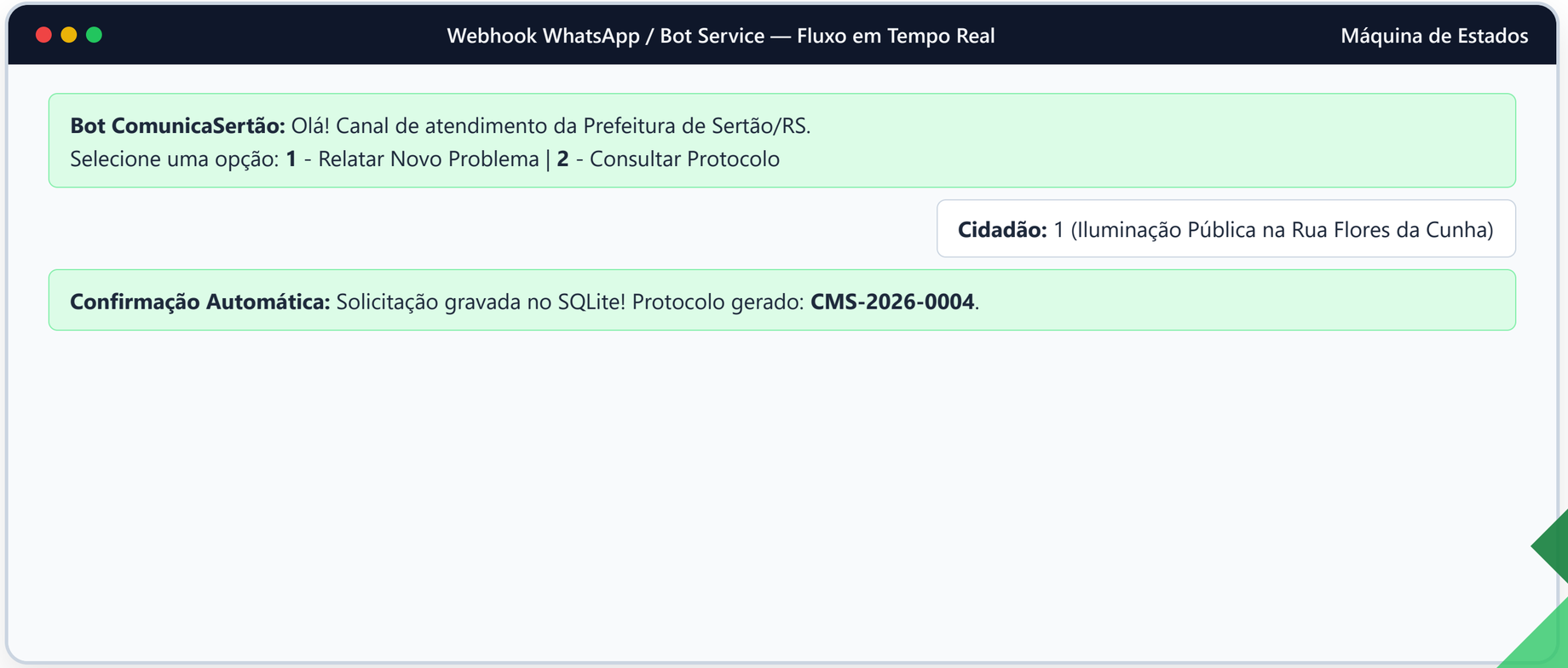
Catálogo OpenAPI e Testes em Tempo Real (/docs)



Geração síncrona da documentação viva, eliminando a dependência de clientes REST externos.

Implementação: Motor Conversacional

Processamento de Webhooks e Geração Sequencial de Protocolos



Triagem lógica automatizada e cálculo de protocolo indexado para auditoria imediata.

Implementação: Controle de Acessos

Governança Institucional e Homologação por Administradores

Tabela funcionarios (comunicasertao.db) — Regras de Segurança — Módulo de Autenticação

Servidor	E-mail Institucional	Cargo	Status
Fabricio de Abreu	fabricio@sertao.rs.gov.br	Administrador	**Aprovado**
Patrick Telo Irschlinger	patrick@sertao.rs.gov.br	Administrador	**Aprovado**
Prefeitura Municipal	prefeitura@sertao.rs.gov.br	Administrador	**Aprovado**
Novo Servidor (Exemplo)	servidor@sertao.rs.gov.br	Servidor Municipal	**Pendente**

Novos cadastros retidos como 'Pendente' com bloqueio HTTP 403 até deliberação de Administrador.

Implementação: Ocorrências e Mídias

Endpoints REST de Mutação e Upload Seguro de Evidências

POST /upload/foto & PATCH /ocorrencias/{id}/status — Serviços de Mídia e Workflow

```
# 1. Upload Seguro de Evidências Fotográficas
ext = os.path.splitext(arquivo.filename)[1].lower()
if ext not in [".jpg", ".jpeg", ".png", ".webp"]:
    raise HTTPException(status_code=400, detail="Formato de imagem inválido")
nome_arquivo = f"foto_{uuid.uuid4().hex[:10]}{ext}"
# Armazenamento e disponibilização via /uploads

# 2. Mutação Atômica de Status (Kanban / Tabela)
ocorrencia.status = payload.status  # "Pendente" -> "Em Andamento" -> "Concluído"
db.commit()
```

Higienização de arquivos via UUID e controle atômico de transições de status no banco SQLite.

Conclusão e Trabalhos Futuros

Resultados Obtidos e Próximos Passos de Engenharia

Deploy em Nuvem

Hospedagem definitiva em servidor VPS com certificado digital SSL/HTTPS e domínio governamental oficial.

Meta Cloud API

Conexão ativa do webhook de produção com o chip telefônico oficial da Prefeitura Municipal de Sertão.

Notificações Ativas

Disparo automático de mensagens no WhatsApp do cidadão a cada avanço no status de seu protocolo cadastrado.